Programación I

TEMA 2. Estructuras de control

Profesores

- Lourdes Borrajo (lborrajo@uvigo.gal)
- Iván Luis (ivan.luis@uvigo.es)

Universidade de Vigo

¿Qué son las estructuras de control?

Las **estructuras de control** permiten que un programa:

- Tome **decisiones**.
- Repita acciones (**bucles**).
- Ejecute instrucciones de forma **ordenada y lógica**.

> ⚠️ Las estructuras de control son la base del control del flujo de un programa.

Tipos comunes

1. **Condicionales** (if, else)
2. **Bucles** (while, for)

Estructura condicional

Una **estructura condicional** es una herramienta en programación que permite que el programa **tome decisiones** y ejecute diferentes bloques de código según se cumpla o no una condición.

> Permite que el programa **realice acciones diferentes** dependiendo de condiciones.

Ejemplos

- Python

```python
edad = 18
if edad >= 18:
    print("Eres mayor de edad")
else:
    print("Eres menor de edad")
```

- Java

```java
int edad = 18;
if (edad >= 18) {
    System.out.println("Eres mayor de edad");
} else {
    System.out.println("Eres menor de edad");
}
```

Cómo tomar decisiones en Python

Python utiliza la estructura `if` para evaluar condiciones y decidir qué código ejecutar.

- Si la condición es verdadera (`True`), ejecuta un bloque de código.
- Si la condición es falsa (`False`), puede ejecutar otro bloque usando `else`.

`if`

`if` ejecuta un bloque de código solo si la condición es verdadera.

```
if condición:
    # código si condición es True
else:
    # código si condición es False
```

`if-elif-else`

```
if condición1:
    # código si condición1 es True
elif condición2:
    # código si condición2 es True
else:
    # código si ninguna condición anterior es True
```

Ejemplo completo

```
nota = 85

if nota >= 90 and nota <= 100:
    print("Excelente")
elif nota >= 80 and nota < 90:
    print("Notable")
elif nota >= 50 and nota < 80:
    print("Aprobado")
elif nota < 50 and nota >= 0:
    print("Suspenso")
else:
    print("Nota fuera de rango")
```

Ejemplo

En una tienda, si eres estudiante, obtienes un descuento. Si no eres estudiante, pagas el precio completo.

Ejemplo en Python

```
es_estudiante = True

if es_estudiante == True:
    print("Descuento aplicado")
else:
    print("Precio completo")
```

```
es_estudiante = True

if es_estudiante:
    print("Descuento aplicado")
else:
    print("Precio completo")
```

Ejemplo

Si tu edad es mayor de 18, puedes entrar. Si es menor de 18, no puedes.

Ejemplo en Python

```
edad = 20
if edad >= 18:
    print("Puedes entrar")
else:
    print("No puedes entrar")
```

Ve a la tienda y ...

... compra una barra de pan. Si tienen huevos, compra una docena.

```
IR A TIENDA

SI HAY HUEVOS ENTONCES
    COMPRAR 12 PANES
FIN SI
```

```
IR A TIENDA

COMPRAR 1 PAN

SI HAY HUEVOS ENTONCES
    COMPRAR 12 HUEVOS

FIN SI
```

Indentación en Python

La indentación es el espacio que se deja al comienzo de una línea.
Define conjunto de instrucciones que se ejecutan juntas bajo una condición, un bucle o una función. **Sintaxis** del lenguaje.

```
edad = 20

if edad >= 18:
    print("Acceso permitido")
    print("Puede entrar")
    print("Puede votar")

print("Fin")
```

> La estructura `if` termina cuando se deja de indentar el código.

Operadores lógicos

Tipo de dato *bool*

- El tipo de dato *bool* puede ser `True` o `False`.

Se considera `False`:

- `False`,
- `None`,
- `0` (int), `0.0` (float), `0j` (complex).
- Colecciones vacías:
  - `""`, `''` (cadenas vacías),
  - `[]` lista vacía,
  - `{}` diccionario vacío,
  - `set()` conjunto vacío,
  - `tuple()` tupla vacía,
  - `range(0)` rango vacío.
- `True` todo lo demás.

Operadores relacionales

Son operadores que permiten **comparar valores**. El resultado es un valor booleano: `True` o `False`.

Operador	Descripción	Ejemplo	Resultado
`==`	Igual a	`5 == 5`	`True`
`!=`	Distinto de	`5 != 5`	`False`
`>`	Mayor que	`7 > 4`	`True`
`<`	Menor que	`8 < 4`	`False`
`>=`	Mayor o igual que	`3 >= 6`	`False`
`<=`	Menor o igual que	`2 <= 3`	`True`

Ejemplo:

```
a = 10
b = 20

print(a == b)  # False
print(a != b)  # True
print(a < b)   # True
print(a > b)   # False
print(a <= b)  # True
print(a >= b)  # False
```

Operadores lógicos

Los **operadores lógicos** permiten **combinar condiciones** y tomar decisiones más complejas en un programa.

Se usan en expresiones booleanas, que dan como resultado `True` (verdadero) o `False` (falso).

Operadores lógicos en Python

Operador	Significado
`and`	Y (ambas deben ser verdaderas)
`or`	O (al menos una debe ser verdadera)
`not`	NO (niega el valor lógico)

Ejemplos en Python

`and` (y)

```
edad = 20
tiene_licencia = True

if edad >= 18 and tiene_licencia == True:
    print("Puedes conducir")
```

`or` (o)

```
dia = "sábado"

if dia == "sábado" or dia == "domingo":
    print("Es fin de semana")
```

`not` (no)

```
es_mayor = False

if not es_mayor:
    print("Es menor de edad")
```

Tabla de verdad del operador `and`, `or`, `not`

La *tabla de verdad* muestra los resultados de una operación lógica según los valores de entrada *A* y *B*.

> Permiten entender cómo se comportan las expresiones lógicas y predecir el resultado de una condición en un programa.

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

Errores comunes al usar `and`, `or`, `not` en Python

Olvidar usar comparaciones completas

- Mal:

```
edad = 20
if edad >= 18 and < 30:
    print("Edad válida")
```

- Bien:

```
edad = 20
if edad >= 18 and edad < 30:
    print("Edad válida")
```

Usar `=` en lugar de `==`

- Mal

```
if activo = True:
    print("Activo")
```

  Error de sintaxis: `=` asigna, no compara.

- Bien

```
    if activo == True:
    print("Activo")
```

  o también

```
    if activo:
    print("Activo")
```

Creer que `and` y `or` devuelven siempre `True` o `False`

- `and` y `or` devuelven el último valor evaluado, no un booleano.

```
resultado = "Hola" and 5
print(resultado)  # Imprime: 5
```

Si necesitas un valor booleano, usar la función `bool(valor_evaluar)`:

```
resultado = bool("Hola" and 5)  # Devuelve: True
print(resultado)  # Imprime: True
```

> ⚠️ `and` devuelve el primer valor falso, o el último valor si todos son verdaderos. `and` busca falsos
> ⚠️ `or` devuelve el primer valor verdadero, o el último valor si todos son falsos. `or` busca verdaderos.

Precedencia de operadores lógicos en Python

¿Qué es la precedencia?

La precedencia determina el orden en que se evalúan los operadores en una expresión con varios operadores lógicos (`not`, `and`, `or`).

Es como en matemáticas:
La expresión `3 + 2 * 4` se evalúa como `3 + (2 * 4)` porque `*` tiene mayor prioridad que `+`.

Orden de precedencia en Python (de mayor a menor)

1. `not`
2. `and`
3. `or`

> Si no se usan paréntesis, Python seguirá este orden para evaluar condiciones.
> Esto puede cambiar el resultado final de una expresión lógica.

Ejemplo sin paréntesis

```
x = True
y = False
z = True

resultado = x or y and not z
print(resultado)
```

Evaluación paso a paso:

1. `not z` → `not True` → `False`
2. `y and not z` → `False and False` → `False`
3. `x or False` → `True or False` → `True`

Resultado: `True`

Ejemplo con paréntesis

```
x = True
y = False
z = True

resultado = (x or y) and not z
print(resultado)
```

Evaluación paso a paso:

1. `x or y` → `True or False` → `True`
2. `not z` → `not True` → `False`
3. `True and False` → `False`

Resultado: `False`

Comparación clara

Expresión	Resultado
`x or y and not z`	`True`
`(x or y) and not z`	`False`

Recomendaciones prácticas

- Usa paréntesis para que el código sea más claro
- Evalúa paso a paso si no estás seguro del orden
- Verifica resultados con pruebas reales